Practical 8bis: Working with Text (Part 2)

The basics of Text Mining and NLP

Table of contents

1 Preamble **2**

2 Setup **2**

3 Illustrative Text Cleaning **6**

4 Applying Normalisation **12**

5 Process the Selected Listings **13**

6 Frequencies and Ngrams **15**

7 Count Vectoriser **17**

8 TF/IDF Vectoriser **29**

9 Word Clouds **31**

10 Latent Dirchlet Allocation **33**

11 Word2Vec **42**

12 Processing the Full File **44**

Part 2 of Practical 8 is *optional* and should only be attempted if Part 1 made sense to you.

1. The first few tasks are about finding important vocabulary (think 'keywords' and 'significant terms') in documents so that you can start to think about what is *distinctive* about documents and groups of documents. **This is quite useful and relatively easier to understand than what comes next!**
2. The second part is about fully-fledged NLP using Latent Direclecht Allocation (topic modelling) and Word2Vec (words embeddings for use in clustering or similarity work).

The later parts are largely complete and ready to run; however, that *doesn't* mean you should just skip over them and think you've grasped what's happening and it will be easy to apply in your own analyses. I would *not* pay as much attention to LDA topic mining since I don't think it's results are that good, but I've included it here as it's still commonly-used in the Digital Humanities and by Marketing folks. Word2Vec is much more powerful and forms the basis of the kinds of advances seen in ChatGPT and other LLMs.

 Connections

Working with text is unquestionably *hard*. In fact, *conceptually* this is probaly the most challenging practical of the term! But data scientists are *always* dealing with text because so much of the data that we collect (even more so thanks to the web) is not only text-based (URLs are text!) but, increasingly, unstructured (social media posts, tags, etc.). So while getting to grips with text is a challenge, it also uniquely positions you with respect to the skills and knowledge that other graduates are offering to employers.

1 Preamble

Parts of this practical have been written using `nltk` and `gensim` — both of which I have **removed** from the sds2026 image to keep the size down and because relatively few students used any of this — but it would be *relatively* easy to rework it to use `spacy`. From what I can see, most programmers tend to use one *or* the other, and the switch wouldn't be hard, other than having to first load the requisite language models. You can read about the models, and note that they are also available in other languages besides English.

2 Setup

Difficulty Level: Low

But this is only because this has been worked out for you. Starting from sctach in NLP is *hard* so people try to avoid it as much as possible.

2.1 Required Modules

Note

Notice that the number of modules and functions that we import is steadily increasing week-on-week, and that for text processing we tend to draw on quite a wide range of utilies! That said, the three most commonly used are: `sklearn` and `nltk`.

Standard libraries we've seen before.

```
import pandas as pd
import geopandas as gpd
import re
import matplotlib.pyplot as plt

# New, but fairly straightforward (for simple things)
from bs4 import BeautifulSoup
```

Vectorisers we will use from the 'big beast' of Python machine learning: Sci-Kit Learn.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

# We don't use this but I point out where you *could*
from sklearn.preprocessing import OneHotEncoder
```

NLP-specific libraries that we will use for tokenisation, lemmatisation, and frequency analysis.

```
import nltk

try:
    from nltk.corpus import wordnet as wn
    from nltk.stem.wordnet import WordNetLemmatizer
    from nltk.corpus import stopwords
    stopwords.words('english')
    lemmatizer = WordNetLemmatizer()
    tokenizer = ToktokTokenizer()
except:
    nltk.download("stopwords")
    nltk.download("punkt_tab")
    nltk.download("averaged_perceptron_tagger_eng")
    from nltk.corpus import stopwords
    from nltk.corpus import wordnet as wn
    from nltk.stem.wordnet import WordNetLemmatizer
    stopwords.words('english')

from nltk.corpus import stopwords
stopword_list = set(stopwords.words('english'))

from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.tokenize.toktok import ToktokTokenizer

from nltk.stem.porter import PorterStemmer
from nltk.stem.snowball import SnowballStemmer

from nltk import ngrams, FreqDist

lemmatizer = WordNetLemmatizer()
tokenizer = ToktokTokenizer()
```

```
[nltk_data] Downloading package stopwords to /home/jovyan/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /home/jovyan/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /home/jovyan/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

Remaining libraries that we'll use for processing and display text data. Most of this relates to

dealing with the various ways that text data cleaning is *hard* because of the myriad formats it comes in.

```
try:
    from wordcloud import WordCloud
except:
    ! pip install wordcloud
import string
import unicodedata
from wordcloud import WordCloud, STOPWORDS
```

This next is just a small utility function that allows us to output Markdown (like this cell) instead of plain text:

```
from IPython.display import display_markdown

def as_markdown(head='', body='Some body text'):
    if head != '':
        display_markdown(f"##### {head}\n\n>{body}\n", raw=True)
    else:
        display_markdown(f">{body}\n", raw=True)

as_markdown('Result!', "Here's my output...")
```

Result!

> Here's my output…

2.2 Loading Data

> ℹ️ Connections
>
> Because I generally want each practical to stand on its own (unless I'm trying to make a *point*), I've not moved this to a separate Python file (e.g. `utils.py`, but in line with what we covered back in the lectures on Functions and Packages, this sort of thing is a good candidate for being split out to a separate file to simplify re-use.

Remember this function from last week? We use it to save downloading files that we already have stored locally. But notice I've made some small changes… what do these do to help the user?

```
from pathlib import Path
from requests import get
from functools import wraps

def check_cache(f):
    @wraps(f)
    def wrapper(src:str, dst:str, min_size=100) -> Path:
        if src.find('?') == -1:
```

```
            url = Path(src)
        else:
            url = Path(src[:src.find('?')])
        fn  = url.name  # Extract the filename
        dsn = Path(f"{dst}/{fn}") # Destination filename
        if dsn.is_file() and dsn.stat().st_size > min_size:
            print(f"+ {dsn} found locally!")
            return(dsn)
        else:
            print(f"+ {dsn} not found, downloading!")
            return(f(src, dsn))
    return wrapper

@check_cache
def cache_data(src:Path, dst:Path) -> str:
    """Downloads a remote file.

    The function sits between the 'read' step of a pandas or geopandas
    data frame and downloading the file from a remote location. The idea
    is that it will save it locally so that you don't need to remember to
    do so yourself. Subsequent re-reads of the file will return instantly
    rather than downloading the entire file for a second or n-th itme.

    Parameters
    ----------
    src : str
        The remote *source* for the file, any valid URL should work.
    dst : str
        The *destination* location to save the downloaded file.

    Returns
    -------
    str
        A string representing the local location of the file.
    """

    # Create any missing directories in dest(ination) path
    # -- os.path.join is the reverse of split (as you saw above)
    # but it doesn't work with lists... so I had to google how
    # to use the 'splat' operator! os.makedirs creates missing
    # directories in a path automatically.
    if not dst.parent.exists():
        dst.parent.mkdir(parents=True, exist_ok=True)

    # Download and write the file
    with dst.open(mode='wb') as file:
        response = get(src)
        file.write(response.content)

    print(' + Done downloading...')
```

```
    return dst.resolve()
```

> **Tip**
>
> For very large *non*-geographic data sets, remember that you can `use_cols` (or `columns` depending on the file type) to specify a subset of columns to load.

Load the main data set:

```
# Load the data sets created in the previous practical
lux    = gpd.read_parquet(Path('data/geo/luxury.geoparquet'))
aff    = gpd.read_parquet(Path('data/geo/affordable.geoparquet'))
bluesp = gpd.read_parquet(Path('data/geo/bluespace.geoparquet'))
```

3 Illustrative Text Cleaning

Now we're going to step through the *parts* of the process that we apply to clean and transform text. We'll do this individually before using a function to apply them *all at once*. To keep things moving along, we're going to take a small sample from the data set.

```
pd.set_option("display.max_colwidth", 250)
sample = bluesp.description.sample(5, random_state=44)
sample
```

```
85363    Peaceful and centrally-located modern studio flat. Fully renovated. River view
28933    River front hidden gem, home-from-home, superbly well-equipped, FREE SUPER FAS
themed,  Ideal for groups/families minimum 3 guests. In the heart of town, excellent tr
82827    Includes 24h concierge, access to a quality gym in the same building, Thames v
56638    A comfortable, fully equipped, first floor, two bedroom apartment with small c
21794    Colourful Bauhaus inspired modern home of British Sculptor with mid-
century and modern furniture and decoration.<br />2 min walk to riverside Embankment be
Name: description, dtype: str
```

3.1 Removing HTML

> **Difficulty level: Moderate**
>
> You'll have seen that tehre are some things that are *not* text in the sample, most notably this stuff: `<br>` and `<br />`. That is raw HTML, the markup language for web pages. To go any further we will need to strip this out. Depending on what you needed to extract from a web page you *can* select different elements by name, class, or id, and do a whole bunch of other things, but here we're just going to crudely strip the tags out.
> You're also going to see a powerful new method: `apply`. Apply simply runs some function against every row or column in the data set depending on how it's called. In this case, we use a `lambda` function (a function without a formal `def`inition) to create a 'Beautiful Soup' HTML parser and then strip the tags from each row.

Hint: you need to need to **get the text** out of the each returned `<p>` and `<div>` element! I'd suggest also commenting this up since there is a *lot* going on on some of these lines of code!

Question

```
cleaned = sample.apply(lambda x: BeautifulSoup(??, 'html.parser').??(" "))
cleaned
```

```
85363    Peaceful and centrally-located modern studio flat. Fully renovated. River view
28933    River front hidden gem, home-from-home, superbly well-equipped, FREE SUPER FAS
themed,  Ideal for groups/families minimum 3 guests. In the heart of town, excellent tr
82827    Includes 24h concierge, access to a quality gym in the same building, Thames v
56638    A comfortable, fully equipped, first floor, two bedroom apartment with small d
21794    Colourful Bauhaus inspired modern home of British Sculptor with mid-
century and modern furniture and decoration. 2 min walk to riverside Embankment betweer
Name: description, dtype: str
```

3.2 Lower Case

Difficulty Level: Low.

Question

```
lower = cleaned.??
lower
```

You should get something like the following:

```
85363    peaceful and centrally-located modern studio flat. fully renovated. river view
28933    river front hidden gem, home-from-home, superbly well-equipped, free super fas
themed,  ideal for groups/families minimum 3 guests. in the heart of town, excellent tr
82827    includes 24h concierge, access to a quality gym in the same building, thames v
56638    a comfortable, fully equipped, first floor, two bedroom apartment with small d
21794    colourful bauhaus inspired modern home of british sculptor with mid-
century and modern furniture and decoration. 2 min walk to riverside embankment betweer
Name: description, dtype: str
```

3.3 Stripping 'Punctuation'

Difficulty level: Hard

This is because you need to understand: 1) why we're *compiling* the regular expression and how to use character classes; and 2) how the NLTK tokenizer differs in its approach to the regex.

3.3.1 Regular Expression Approach

We want to clear out punctuation using a regex that takes advantage of the `[...]` (character class) syntax. The really tricky part is remembering how to specify the 'punctuation' when some of that punctuation has 'special' meanings in a regular expression context. For instance, `.` means 'any character', while `[` and `]` mean 'character class'. So this is another *escaping* problem and it works the *same* way it did when we were dealing with the Terminal…

Hints: some other factors…

1. You will want to match more than one piece of punctuation at a time, so I'd suggest add a `+` to your pattern.
2. You will need to look into *metacharacters* for creating a kind of 'any of the characters *in this class*' bag of possible matches.

Question

```
pat = re.compile(r'[???]+')
subbed = lower.apply(lambda x: pat.sub('',x))
```

Depending on how thorough you are, you should get something like this:

```
85363    peaceful and centrallylocated modern studio flat fully renovated river view ne
28933    river front hidden gem homefromhome superbly wellequipped free super fast fibr
82827    includes 24h concierge access to a quality gym in the same building thames vie
56638    a comfortable fully equipped first floor two bedroom apartment with small outd
21794    colourful bauhaus inspired modern home of british sculptor with midcentury and
Name: description, dtype: str
```

3.3.2 Tokenizer

The other way to do this, which is probably *easier*, is to draw on the tokenizers already provided by NLTK. For our purposes `word_tokenize` is probably fine, but depending on your needs there are other options and you can also write your own.

```
nltk.download('punkt')
nltk.download('wordnet')
from nltk.tokenize import word_tokenize
tokens = lower.apply(word_tokenize)
tokens
```

```
[nltk_data] Downloading package punkt to /home/jovyan/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /home/jovyan/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
85363    [peaceful, and, centrally-located, modern, studio, flat, ., fully, renovated,
28933    [river, front, hidden, gem, ,, home-from-home, ,, superbly, well-
equipped, ,, free, super, fast, fibre, wifi, centre, of, london, ,, vintage-
themed, ,, ideal, for, groups/families, minimum, 3, guests, ., in, the, heart, of, town
```

```
82827    [includes, 24h, concierge, ,, access, to, a, quality, gym, in, the, same, buil
56638    [a, comfortable, ,, fully, equipped, ,, first, floor, ,, two, bedroom, apartme
21794    [colourful, bauhaus, inspired, modern, home, of, british, sculptor, with, mid-
century, and, modern, furniture, and, decoration, ., 2, min, walk, to, riverside, embar
Name: description, dtype: object
```

> ⚠ Not Equivalent!
>
> Notice that these aren't giving you the same result!
> For our purposes it is easier in the subsequent stages to work with the `list` output from tokenize rather than the string output from `re.sub`, but if you needed to convert tokenise back to a string you could do it this way:
>
> ```
> tokens = lower.apply(word_tokenize).apply(lambda x: ' '.join(x))
> ```
>
> The other way around would be:
>
> ```
> lower.apply(lambda x: pat.sub('',x)).str.split(' ')
> ```

3.4 Stopword Removal

> ⚠ Difficulty Level: Moderate
>
> You need to remember how list comprehensions work to use the `stopword_list`.

```
stopword_list = set(stopwords.words('english'))
print(stopword_list)
```

```
{'ve', "it'll", 'again', 'no', 'being', 'what', 'few', 'had', 're', 'those', 'were', 's
```

Question

```
passthru = []
for t in tokens[2:4]:
    passthru.append([x for x in t if ?? and len(x) > 1])

for p in passthru:
    as_markdown("Stopwords removed:", p)
```

3.5 Lemmatisation vs Stemming

> Difficulty level: Low.

```
from nltk.stem.porter import PorterStemmer
from nltk.stem.snowball import SnowballStemmer
from nltk.stem.wordnet import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize('monkeys'))
print(lemmatizer.lemmatize('cities'))
print(lemmatizer.lemmatize('complexity'))
print(lemmatizer.lemmatize('Reades'))
```

```
monkey
city
complexity
Reades
```

```
stemmer = SnowballStemmer(language='english')
print(stemmer.stem('monkeys'))
print(stemmer.stem('cities'))
print(stemmer.stem('complexity'))
print(stemmer.stem('Reades'))
```

```
monkey
citi
complex
read
```

```
lemmatizer = WordNetLemmatizer()
stemmer    = SnowballStemmer(language='english')

# This would be better if we passed in a PoS (Part of Speech) tag as well,
# but processing text for parts of speech is *expensive* and for the purposes
# of this tutorial, not necessary.
lemmas  = passthru.apply(lambda x: [lemmatizer.lemmatize(t) for t in x])
stemmed = passthru.apply(lambda x: [stemmer.stem(t) for t in x])

as_markdown("Lemmatised", '\n\n>'.join(lemmas.str.join(' ').to_list()))
print("\n\n")
as_markdown("Stemmed", '\n\n>'.join(stemmed.str.join(' ').to_list()))
```

Lemmatised

> peaceful centrally-located modern studio flat fully renovated river view next victoria pimlico train station opposite vauxhall bridge minute walk vauxhall 30 minute london gatwick via train 57 square metre i.e 614 square foot kitchen wifi amenity including washing machine luxury bedding large wardrobe lot stirage luxurious modern building 24h security concierge high ceiling air conditioned excellent condition

river front hidden gem home-from-home superbly well-equipped free super fast fibre wifi centre london vintage-themed ideal groups/families minimum guest heart town excellent transport links.comfortable cosy apartment quiet safe peaceful neighbourhood rich history character shop bar restaurant ideal river trips/walks sightseeing fully stocked kitchen breakfast good welcome treat

includes 24h concierge access quality gym building thames view river 20 metre away close one best pub london trafalgar tavern famous greenwich observatory hill top view london old royal naval college o2 arena event 10 min north greenwich jubilee line commuting 15 min walk cutty sark dlr either friend life apartment live

comfortable fully equipped first floor two bedroom apartment small outdoor seating area situated 10 minute walk kew garden national archive close river thames excellent public transport ten minute walk kew garden underground station numerous restaurant pub cafe grocery store within short walking distance

colourful bauhaus inspired modern home british sculptor mid-century modern furniture decoration min walk riverside embankment lambeth palace vauxhall station expect train going past front house- ideal traveller couple family looking explore london vauxhall station 10min walk- bus 344 77 taking 6min cafe bar beaconsfield gallery /cafe /restaurant damien hirst newport street gallery west end 32min walk

Stemmed

peac centrally-loc modern studio flat fulli renov river view next victoria pimlico train station opposit vauxhal bridg minut walk vauxhal 30 minut london gatwick via train 57 squar metr i.e 614 squar feet kitchen wifi amen includ wash machin luxuri bed larg wardrob lot stirag luxuri modern build 24h secur concierg high ceil air condit excel condit

river front hidden gem home-from-hom superbl well-equip free super fast fibr wifi centr london vintage-them ideal groups/famili minimum guest heart town excel transport links.comfort cosi apart quiet safe peac neighbourhood rich histori charact shop bar restaur ideal river trips/walk sightse fulli stock kitchen breakfast good welcom treat

includ 24h concierg access qualiti gym build thame view river 20 metr away close one best pub london trafalgar tavern famous greenwich observatori hill top view london old royal naval colleg o2 arena event 10 min north greenwich jubile line commut 15 min walk cutti sark dlr either friend live apart live

comfort fulli equip first floor two bedroom apart small outdoor seat area situat 10 minut walk kew garden nation archiv close river thame excel public transport ten minut walk kew garden underground station numer restaur pub cafe groceri store within short walk distanc

colour bauhaus inspir modern home british sculptor mid-centuri modern furnitur decor min walk riversid embank lambeth palac vauxhal station expect train go past front house- ideal travel coupl famili look explor london vauxhal station 10min walk- buse 344 77 take 6min cafe bar beaconsfield galleri /cafe /restaur damien hirst newport street galleri west end 32min walk

4 Applying Normalisation

The above approach is fairly hard going since you need to loop through every list element applying these changes one at a time. Instead, we could convert the column to a corpus (or use pandas `apply`) together with a function imported from a library to do the work.

4.1 Downloading the Custom Module

In a Jupyter notebook, this code allows us to edit and reload the library dynamically:

```
%load_ext autoreload
%autoreload 2
```

> Difficulty level: Low.

This custom module is not perfect, but it gets the job done… mostly and has some additional features that you could play around with for a final project (e.g. `detect_entities` and `detect_acronyms`).

```
try:
    from textual import *
except:
    try:
        from unidecode import unidecode
    except:
        ! pip install unidecode
    import urllib.request
    host  = 'http://orca.casa.ucl.ac.uk'
    turl  = f'{host}/~jreades/__textual__.py'
    tdirs = Path('textual')
    tpath = Path(tdirs / '__init__.py')

    if not tpath.exists():
        tdirs.mkdir(parents=True, exist_ok=True)
        urllib.request.urlretrieve(turl, tpath)
    from textual import *
```

```
All NLTK libraries installed...
```

4.2 Importing the Custom Module

> Difficulty Level: Low.
>
> But only because you didn't have to write the module! However, the questions could be hard…

```
normalised = sample.apply(normalise_document, remove_digits=True)
as_markdown('Normalised', '\n\n>'.join(normalised.to_list()))
```

Normalised

> peaceful centrallylocated modern studio flat . fully renovate . river view . next victoria pimlico train station opposite vauxhall bridge . minute walk vauxhall . minute london gatwick train . square metre . square foot . kitchen wifi amenity include wash machine luxury bedding . large wardrobe stirage . luxurious modern building security concierge . high ceiling . condition . excellent condition .
>
> river front hidden homefromhome superbly wellequipped free super fast fibre wifi centre london vintagethemed ideal groupsfamilies minimum guest . heart town excellent transport links.comfortable cosy apartment quiet safe peaceful neighbourhood rich history character . shops restaurant . ideal river tripswalks sightsee . fully stock kitchen . breakfast good welcome treat .
>
> includes concierge access quality building thames view . river metre away . close best london trafalgar tavern famous greenwich observatory . hill view london . royal naval college arena . event . north greenwich jubilee line commute walk cutty sark either friend live apartment live
>
> comfortable fully equip first floor bedroom apartment small outdoor seat area . situated minute walk gardens national archive close river thames . excellent public transport minute walk gardens underground station . numerous restaurant cafe grocery store within short walk distance .
>
> colourful bauhaus inspire modern home british sculptor midcentury modern furniture decoration . walk riverside embankment lambeth palace vauxhall station expect train past front house ideal traveller couple family look explore london . vauxhall station walk take cafe beaconsfield gallery cafe restaurant damien hirs newport street gallery . west walk .

```
help(normalise_document)
```

> Stop!
>
> Beyond this point, we are moving into Natural Language Processing. If you are already struggling with regular expressions, I would recommend *stopping here*. You can come back to revisit the NLP components and creation of word clouds later.

5 Process the Selected Listings

> Difficulty level: Low, but you'll need to be patient!

Notice the use of `%%time` here – this will tell you how long each block of code takes to complete. It's a really useful technique for reminding *yourself* and others of how long something might

take to run. I find that with NLP this is particularly important since you have to do a *lot* of processing on each document in order to normalise it.

> **Tip**
>
> Notice how we can change the default parameters for `normalise_document` even when using `apply`, but that the syntax is different. So whereas we'd use `normalise_document(doc, remove_digits=True)` if calling the function directly, here it's `.apply(normalise_document, remove_digits=True)`!

Question

```
%%time
# I get about 2 seconds on a M2 Mac
bluesp['description_norm'] = bluesp.???.apply(???, remove_digits=True)
```

5.1 Select and Tokenise

> Difficulty level: Low, except for the double list-comprehension.

5.1.1 Select and Extract Corpus

See useful tutorial here. Although we shouldn't have any empty descriptions, by the time we've finished normalising the textual data we may have *created* some empty values and we need to ensure that we don't accidentally pass a NaN to the vectorisers and frequency distribution functions.

```
# This makes it easy to go back and re-run
# the analysis with other data subsets.
srcdf = bluesp
```

> **Coding Tip**
>
> Notice how you only need to change the value of the variable here to try any of the different selections we did above? This is a simple kind of parameterisation somewhere between a function and hard-coding everything.

```
corpus = srcdf.description_norm.fillna(' ').values
print(corpus[0:3])
```

```
<ArrowStringArray>
[
 'spacious  light quiet apartment combine best charm london  juxtaposition . located ga
Length: 3, dtype: str
```

5.1.2 Tokenise

There are different forms of tokenisation and different algorithms will expect differing inputs. Here are two:

```
sentences = [nltk.sent_tokenize(text) for text in corpus]
words     = [[nltk.tokenize.word_tokenize(sentence)
                for sentence in nltk.sent_tokenize(text)]
                for text in corpus]
```

Notice how this has turned every sentence into an array and each document into an array of arrays:

```
print(f"Sentences 0: {sentences[0]}")
print()
print(f"Words 0: {words[0]}")
```

```
Sentences 0: ['house garden close thames river .', 'walk  private road river nearby .',

Words 0: [['house', 'garden', 'close', 'thames', 'river', '.'], ['walk', 'private', 'ro
```

6 Frequencies and Ngrams

⚠ Difficulty level: Moderate.

One new thing you'll see here is the `ngram`: ngrams are 'simply' pairs, or triplets, or quadruplets of words. You may come across the terms unigram (`ngram(1,1)`), bigram (`ngram(2,2)`), trigram (`ngram(3,3)`)… typically, you will rarely find anything beyond trigrams, and these present real issues for text2vec algorithms because the embedding for `geographical`, `information`, and `systems` is *not* the same as for `geographical information systetms`.

6.0.1 Build Frequency Distribution

Build counts for ngram range 1..3:

```
fcounts = dict()

# Here we replace all full-stops... can you think why we might do this?
data = nltk.tokenize.word_tokenize(' '.join([text.replace('.','') for text in corpus

for size in 1, 2, 3:
    fdist = FreqDist(ngrams(data, size))
    print(fdist)
    # If you only need one note this: https://stackoverflow.com/a/52193485/4041902
    fcounts[size] = pd.DataFrame.from_dict({f'Ngram Size {size}': fdist})
```

```
<FreqDist with 1796 samples and 13228 outcomes>
<FreqDist with 7653 samples and 13227 outcomes>
<FreqDist with 9977 samples and 13226 outcomes>
```

6.0.2 Output Top-n Ngrams

And output the most common ones for each ngram range:

```
for dfs in fcounts.values():
    print(dfs.sort_values(by=dfs.columns.values[0], ascending=False).head(10))
    print()
```

```
           Ngram Size 1
walk                481
london              341
minute              282
station             261
river               247
view                189
apartment           179
thames              157
close               147
room                147

                  Ngram Size 2
minute  walk               178
river   view                83
central london              80
river   thames              73
canary  wharf               59
walk    river               43
        waterloo            38
station walk                36
walk    distance            35
thames  river               33

                            Ngram Size 3
walk    river    thames                33
fully   equip    kitchen               19
minute  walk     river                 19
station minute   walk                  19
within  walk     distance              17
walk    thames   river                 16
        waterloo station               16
close   river    thames                15
minute  walk     waterloo              15
cosy    bedroom  apartment             13
```

Questions

1. Can you think why we don't care about punctuation for frequency distributions and n-grams?
2. Do you understand what n-grams *are*?

7 Count Vectoriser

Difficulty level: Low, but the output needs some thought!

This is a big foray into sklearn (sci-kit learn) which is the main machine learning and clustering module for Python. For processing text we use a *vectorisers* to convert terms to a vector representation. We're doing this on the smallest of the derived data sets because these processes can take a while to run and generate *huge* matrices (remember: one row and one column for each term!).

7.1 Fit the Vectoriser

```
cvectorizer = CountVectorizer(ngram_range=(1,3))
cvectorizer.fit(corpus)
```

---

ngram_range	ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different word n-grams or char n-grams to be extracted. All values of n such such that min_n <= n <= max_n will be used. For example an "ngram_range" of "(1, 1)" means only unigrams, "(1, 2)" means unigrams and bigrams, and "(2, 2)" means only bigrams. Only applies if "analyzer" is not callable.	(1, ...)

	input input: {'filename', 'file', 'content'}, default='content' - If ‘'filename'‘, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If ‘'file'‘, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory. - If ‘'content'‘, the input is expected to be a sequence of items that can be of type string or byte.	'content'
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given ‘encoding‘. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	'strict'

	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:‘unicodedata.normal-ize‘. lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing. preprocessor preprocessor: callable, default=None Override the preprocessing (strip_accents and lowercase) stage while preserving the tokenizing and n-grams generation steps. Only applies if “analyzer“ is not callable. tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if “analyzer == 'word'“.	None True None None

	stop_words stop_words: {'english'}, list, default=None If 'english', a built-in stop word list for English is used. There are several known issues with 'english' and you should consider an alternative (see :ref:‘stop_words‘). If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if “analyzer == 'word'“. If None, no stop words will be used. In this case, setting ‘max_df‘ to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	None
	token_pattern token_pattern: str or None, default=r”(?u)\\b\\w\\w+\\b” Regular expression denoting what constitutes a ”token”, only used if “analyzer == 'word'“. The default regexp select tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'

	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word n-gram or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if “input“ is “filename“ or “file“, the data is first read from the file and then passed to the given callable analyzer.	'word'
	max_df max_df: float in range [0.0, 1.0] or int, default=1.0 When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1.0

	min_df min_df: float in range [0.0, 1.0] or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	max_features max_features: int, default=None If not None, build a vocabulary that only consider the top ‘max_features‘ ordered by term frequency across the corpus. Otherwise, all features are used. This parameter is ignored if vocabulary is not None.	None
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents. Indices in the mapping should not be repeated and should not have any gap between 0 and the largest index.	None

	binary binary: bool, default=False If True, all non zero counts are set to 1. This is useful for discrete probabilistic models that model binary events rather than integer counts.	False
	dtype dtype: dtype, default=np.int64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.int64'>

Name	Type	Value
fixed_vocabulary_ fixed_vocabulary_: bool True if a fixed vocabulary of term to indices mapping is provided by the user.	bool	False
vocabulary_ vocabulary_: dict A mapping of terms to feature indices.	dict	{'aback': 0, 'ab...us': 1, 'ab...nt': 2, 'abba': 3, ...}

aback
aback spacious
aback spacious apartment
abba
abba voyage
abba voyage load
abba voyage walk
abba walk
abba walk stratford
abbey
abbey find
abbey find conveniently
abbey imperial
abbey imperial museum
abbeywood
abbeywood shooping
abbeywood shooping mall
abbeywood station
abbeywood station take

abbeywood trainstation
abbeywood trainstation elizabeth
able
able previous
able previous review
abode
abode visitor
abode visitor london
abroad
abroad occasionally
abroad occasionally item
absolutely
absolutely enormous
absolutely enormous room
ac
ac throughout
ac throughout free
academic
academic couple
academic couple child
academic family
academic family share
accept
accept ladies
accept ladies cosy
access
access amenity
access amenity include
access around
access around london
access arrange
access arrange pick
access available
access available couples
access buses
access buses west
access canal
access canal view
access canary
access canary wharf
access central
access central london
access city
access city center
access city london
access close
access close tube
access comfy
access comfy mx
access elizabeth
access elizabeth line

access everything
access everything centrally
access everything share
access facility
access facility flat
access flat
access flat entire
access gallions
access gallions reach
access gym
access gym concierge
access heathrow
access heathrow airport
access ideal
access ideal convenient
access king
access king size
access london
access london attraction
access london attractions
access london comfortably
access london enthusiast
access lounge
access lounge floor
access min
access min walk
access mode
access mode public
access multiple
access multiple transport

7.2 Brief Demonstration

Once we've fit the vectoriser, we can find the integer associated with any word in the corpus and can also ask how many times it occurs:

```
pd.options.display.max_colwidth=750

# Note that the term can be a unigram, a bigram, or a trigram!
term = 'minute walk'
# Find the vocabulary mapping for the term
print(f"Vocabulary mapping for '{term}' is {cvectorizer.vocabulary_[term]}")

# How many times is it in the data
print(f"Found {srcdf.description_norm.str.contains(term).sum():,} rows containing '{

# Print the descriptions containing the term
for i, x in enumerate(srcdf[srcdf.description_norm.str.contains(term)].description_n
    as_markdown(f"{term}: {i}",x)
```

```
    if i > 4: break
```

```
Vocabulary mapping for 'minute walk' is 9828
Found 106 rows containing 'minute walk'
```

minute walk: 0

please read things note comfortable clean bright brand flat east london minute central london tube quite central great transport link major london attraction minute walk river park undergroundtubedlr station supermarket docklands stratford olympic stadium westfield shopping centre . enjoy brick lane indian restaurant spitalfields market colombian flower market historical whitechapel .

minute walk: 1

comfortable fairly flat east london travel zone minute central london quite central great transport link major london attraction minute walk river park undergroundtube station minute supermarket minute docklands stratford olympic stadium westfield shopping centre . enjoy brick lane indian restaurant spitalfields market colombian flower market historical whitechapel .

minute walk: 2

cosy bedroom apartment ninth floor amazing river view minute pimlico station heart city . zone . bigben buckingham palace hyde park chelsea minute walk distance .

minute walk: 3

a perfect central london location zone comfortable cosy bedroom apartment locate central location london . the apartment also locate minute walk elephant castle station also close station apartment . one benefit walk waterloo . london . whiten minute . many travel easy fast

minute walk: 4

perfect claphambattersea modern decor . brand kitchenbathroom minute walk clapham junction station free parking green space view close thames river path battersea park contact . delight hear

minute walk: 5

minutes walk tate modern globe theatre southbank borough market river host attraction . super location transport next southwark metro station less minute walk waterloo london bridge

7.3 Transform the Corpus

You can only *tranform* the entire corpus *after* the vectoriser has been fitted. There is an option to `fit_transform` in one go, but I wanted to demonstrate a few things here and some vectorisers are don't support the one-shot fit-and-transform approach. **Note the type of the transformed corpus**:

```
cvtcorpus = cvectorizer.transform(corpus)
cvtcorpus # cvtcorpus for count-vectorised transformed corpus
```

```
<Compressed Sparse Row sparse matrix of dtype 'int64'
    with 36754 stored elements and shape (333, 18485)>
```

7.3.1 Single Document

Here is the **first** document from the corpus:

```
doc_df = pd.DataFrame(cvtcorpus[0].T.todense(),
                      index=cvectorizer.get_feature_names_out(), columns=["Counts"]
                     ).sort_values('Counts', ascending=False)
doc_df.head(10)
```

	Counts
walk	3
river	2
thames	2
playground	1
line	1
line underground walk	1
house	1
line underground	1
playground walk distance	1
playground walk	1

7.3.2 Transformed Corpus

```
cvdf = pd.DataFrame(data=cvtcorpus.toarray(),
                        columns=cvectorizer.get_feature_names_out())
print(f"Raw count vectorised data frame has {cvdf.shape[0]:,} rows and {cvdf.shape[1
cvdf.iloc[0:3,0:7]
```

```
Raw count vectorised data frame has 333 rows and 18,485 columns.
```

	aback	aback spacious	aback spacious apartment	abba	abba voyage	abba voyage load	abba voy
0	0	0	0	0	0	0	0

	aback	aback spacious	aback spacious apartment	abba	abba voyage	abba voyage load	abba voy
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0

```
cvdf.iloc[-3:,-7:]
```

	zone recently refurbish	zone tube	zone tube station	zone victoria	zone victoria line	zone wellf
330	0	0	0	0	0	0
331	0	0	0	0	0	0
332	0	0	0	0	0	0

Questions

1. Why is the single document view a list of terms and counts?
2. Why is the corpus view a table of terms (columns) and integers (rows)?

7.3.3 Filter Low-Frequency Words

These are likely to be artefacts of text-cleaning or human input error. As well, if we're trying to look across an entire corpus then we might not want to retain words that only appear in a couple of documents.

Let's start by getting the *column* sums:

```
sums = cvdf.sum(axis=0)
print(f"There are {len(sums):,} terms in the data set.")
sums.head()
```

```
There are 18,485 terms in the data set.

aback                      1
aback spacious             1
aback spacious apartment   1
abba                       4
abba voyage                3
dtype: int64
```

Remove columns (i.e. terms) appearing in less than 1% of documents. You can do this by thinking about what the shape of the data frame means (rows and/or columns) and how you'd get 1% of that!

Question

```
filter_terms = sums >= cvdf.shape[0] * ???
```

Now see how we can use this to strip out the columns corresponding to low-frequency terms:

```
fcvdf = cvdf.drop(columns=cvdf.columns[~filter_terms].values)
print(f"Filtered count vectorised data frame has {fcvdf.shape[0]:,} rows and {fcvdf.
fcvdf.iloc[0:3,0:7]
```

```
Filtered count vectorised data frame has 333 rows and 1,623 columns.
```

	abba	abbeywood	access	access buses	access buses west	access central	access central london
0	0	0	1	0	0	1	1
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0

```
fcvdf.sum(axis=0)
```

```
abba                        4
abbeywood                   4
access                     63
access buses                4
access buses west           4
                           ..
zone comfortable            7
zone comfortable cosy       7
zone london                 5
zone recently               5
zone recently refurbish     5
Length: 1623, dtype: int64
```

We're going to pick this up again in Task 7.

Questions

- Can you explain what `doc_df` contains?
- What does `cvdf` contain? Explain the rows and columns.
- What is the function of `filter_terms`?

8 TF/IDF Vectoriser

> ⚠ Difficulty level: Moderate
>
> But only if you want to understand how `max_df` and `min_df` work!

8.1 Fit and Transform

```
tfvectorizer = TfidfVectorizer(use_idf=True, ngram_range=(1,3),
                               max_df=0.75, min_df=0.01) # <-- these matter!
tftcorpus    = tfvectorizer.fit_transform(corpus) # TF-transformed corpus
```

8.2 Single Document

```
doc_df = pd.DataFrame(tftcorpus[0].T.todense(), index=tfvectorizer.get_feature_names
doc_df.sort_values('Weights', ascending=False).head(10)
```

	Weights
along	0.253207
close thames river	0.253207
path	0.236829
close thames	0.236829
district line	0.236829
river walk	0.230329
access central	0.224595
access central london	0.224595
district	0.219466
gardens	0.219466

8.3 Transformed Corpus

```
tfidf = pd.DataFrame(data=tftcorpus.toarray(),
                     columns=tfvectorizer.get_feature_names_out())
print(f"TF/IDF data frame has {tfidf.shape[0]:,} rows and {tfidf.shape[1]:,} columns
tfidf.head()
```

```
TF/IDF data frame has 333 rows and 1,589 columns.
```

	abba	abbeywood	access	access buses	access buses west	access central	access central londo
0	0.0	0.0	0.134744	0.0	0.0	0.224595	0.224595
1	0.0	0.0	0.000000	0.0	0.0	0.000000	0.000000
2	0.0	0.0	0.000000	0.0	0.0	0.000000	0.000000
3	0.0	0.0	0.000000	0.0	0.0	0.000000	0.000000
4	0.0	0.0	0.000000	0.0	0.0	0.000000	0.000000

Questions

- What does the TF/IDF score *represent*?
- What is the role of `max_df` and `min_df`?

9 Word Clouds

9.1 For Counts

Difficulty level: Easy!

```
fcvdf.sum().sort_values(ascending=False)
```

```
walk                              481
london                            341
minute                            282
station                           261
river                             247
                                 ...
shops restaurants                   4
shops restaurants battersea         4
ideally                             4
idea onto paper                     4
abba                                4
Length: 1623, dtype: int64
```

```
ff = Path('RobotoSlab-VariableFont_wght.ttf')
dp = Path('/home/jovyan/.fonts/')
tp = Path(Path.home(),'Library','Fonts')
if tp.exists():
    fp = tp / ff
else:
    fp = dp / ff
```

```
f,ax = plt.subplots(1,1,figsize=(5,5))
plt.gcf().set_dpi(300)
Cloud = WordCloud(
    background_color="white",
    max_words=75,
    font_path=fp
).generate_from_frequencies(fcvdf.sum())
ax.imshow(Cloud)
ax.axis("off");
#plt.savefig("Wordcloud 1.png")
```



9.2 For TF/IDF Weighting

> Difficulty level: Low, but you'll need to be patient!

```
tfidf.sum().sort_values(ascending=False)
```

```
walk                          22.653792
minute                        17.585253
london                        17.023854
river                         14.183239
view                          13.583084
                                 ...
common easy access             0.330871
common easy                    0.330871
close shops restaurants        0.330871
close river river              0.330871
private roof terrace           0.330871
Length: 1589, dtype: float64
```

```
f,ax = plt.subplots(1,1,figsize=(5,5))
plt.gcf().set_dpi(300)
Cloud = WordCloud(
    background_color="white",
    max_words=100,
    font_path=fp
).generate_from_frequencies(tfidf.sum())
ax.imshow(Cloud)
ax.axis("off");
#plt.savefig("Wordcloud 2.png")
```



Questions

- What does the `sum` represent for the count vectoriser?
- What does the `sum` represent for the TF/IDF vectoriser?

10 Latent Dirchlet Allocation

> **Tip**
>
> I would give this a *low* priority. It's a commonly-used method, but on small data sets it really isn't much use and I've found its answers to be… unclear… even on large data sets.

Adapted from this post on doing LDA using sklearn. Most other examples use the `gensim` library.

```
# Notice change to ngram range
# (try 1,1 and 1,2 for other options)
vectorizer = CountVectorizer(ngram_range=(1,2))
```

10.1 Calculate Topics

```
vectorizer.fit(corpus)
tcorpus = vectorizer.transform(corpus) # tcorpus for transformed corpus

LDA = LatentDirichletAllocation(n_components=3, random_state=42) # Might want to exp
LDA.fit(tcorpus)
```

	n_components n_components: int, default=10 Number of topics. .. versionchanged:: 0.19 “n_topics“ was renamed to “n_components“	3
	random_state random_state: int, RandomState instance or None, default=None Pass an int for reproducible results across multiple function calls. See :term:‘Glossary <random_state>‘.	42
	doc_topic_prior doc_topic_prior: float, default=None Prior of document topic distribution ‘theta‘. If the value is None, defaults to ‘1 / n_components‘. In [1]_, this is called ‘alpha‘.	None
	topic_word_prior topic_word_prior: float, default=None Prior of topic word distribution ‘beta‘. If the value is None, defaults to ‘1 / n_components‘. In [1]_, this is called ‘eta‘.	None

	learning_method learning_method: {'batch', 'online'}, default='batch' Method used to update ‘_component‘. Only used in :meth:‘fit‘ method. In general, if the data size is large, the online update will be much faster than the batch update. Valid options: - 'batch': Batch variational Bayes method. Use all training data in each EM update. Old ‘components_‘ will be overwritten in each iteration. - 'online': Online variational Bayes method. In each EM update, use mini-batch of training data to update the “components_“ variable incrementally. The learning rate is controlled by the “learning_decay“ and the “learning_offset“ parameters. .. versionchanged:: 0.20 The default learning method is now “”batch”“.	'batch'
	learning_decay learning_decay: float, default=0.7 It is a parameter that control learning rate in the online learning method. The value should be set between (0.5, 1.0] to guarantee asymptotic convergence. When the value is 0.0 and batch_size is “n_samples“, the update method is same as batch learning. In the literature, this is called kappa.	0.7

	learning_offset learning_offset: float, default=10.0 A (positive) parameter that downweights early iterations in online learning. It should be greater than 1.0. In the literature, this is called tau_0.	10.0
	max_iter max_iter: int, default=10 The maximum number of passes over the training data (aka epochs). It only impacts the behavior in the :meth:‘fit‘ method, and not the :meth:‘partial_fit‘ method.	10
	batch_size batch_size: int, default=128 Number of documents to use in each EM iteration. Only used in online learning.	128
	evaluate_every evaluate_every: int, default=-1 How often to evaluate perplexity. Only used in ‘fit‘ method. set it to 0 or negative number to not evaluate perplexity in training at all. Evaluating perplexity can help you check convergence in training process, but it will also increase total training time. Evaluating perplexity in every iteration might increase training time up to two-fold.	-1

	total_samples total_samples: int, default=1e6 Total number of documents. Only used in the :meth:‘partial_fit‘ method.	1000000.0
	perp_tol perp_tol: float, default=1e-1 Perplexity tolerance. Only used when “evaluate_every“ is greater than 0.	0.1
	mean_change_tol mean_change_tol: float, default=1e-3 Stopping tolerance for updating document topic distribution in E-step.	0.001
	max_doc_update_iter max_doc_update_iter: int, default=100 Max number of iterations for updating document topic distribution in the E-step.	100
	n_jobs n_jobs: int, default=None The number of jobs to use in the E-step. “None“ means 1 unless in a :obj:‘joblib.parallel_backend‘ context. “-1“ means using all processors. See :term:‘Glossary <n_jobs>‘ for more details.	None
	verbose verbose: int, default=0 Verbosity level.	0

Name	Type	Value
bound_ bound_: float Final perplexity score on training set.	float64	4296

components_ components_: ndarray of shape (n_components, n_features) Variational parameters for topic word distribution. Since the complete conditional for topic word distribution is a Dirichlet, “components_[i, j]“ can be viewed as pseudocount that represents the number of times word ‘j‘ was assigned to topic ‘i‘. It can also be viewed as distribution over the words for each topic after normalization: “model.components_ / model.compo- nents_.sum(axis=1)[:, np.newaxis]“.	ndarray[float64](3, 9148)	[[0.33,0.33,1.33,...,1.33,0.33,0.33], [1.33,1.33,2.34,...,0.33,0.33,1.33], [0.33,0.33,1.33,...,0.33,1.33,0.33]]
doc_topic_prior_ doc_topic_prior_: float Prior of document topic distribution ‘theta‘. If the value is None, it is ‘1 / n_components‘.	float	0.3333
exp_dirichlet_component_ exp_dirichlet_component_: ndarray of shape (n_components, n_features) Exponential value of expectation of log topic word distribution. In the literature, this is ‘exp(E[log(beta)])‘.	ndarray[float64](3, 9148)	[[0.,0.,0.,...,0.,0.,0.], [0.,0.,0.,...,0.,0.,0.], [0.,0.,0.,...,0.,0.,0.]]
n_batch_iter_ n_batch_iter_: int Number of iterations of the EM step.	int	11
n_features_in_ n_features_in_: int Number of features seen during :term:‘fit‘. .. versionadded:: 0.24	int	9148

n_iter_ n_iter_: int Number of passes over the dataset.	int	10
random_state_ random_state_: RandomState instance RandomState instance that is generated either from a seed, the random number generator or by ‘np.random‘.	RandomState	Random-State(M...0xFFFF19FCE540
topic_word_prior_ topic_word_prior_: float Prior of topic word distribution ‘beta‘. If the value is None, it is ‘1 / n_components‘.	float	0.3333

latentdirichletallocation0
latentdirichletallocation1
latentdirichletallocation2

```
first_topic = LDA.components_[0]
top_words = first_topic.argsort()[-25:]

for i in top_words:
    print(vectorizer.get_feature_names_out()[i])
```

```
walk waterloo
also
zone
locate
central london
city
stay
restaurant
kitchen
thames
bedroom
location
flat
room
close
waterloo
river
```

```
view
central
apartment
station
minute walk
london
minute
walk
```

```
for i,topic in enumerate(LDA.components_):
    as_markdown(f'Top 10 words for topic #{i}', ', '.join([vectorizer.get_feature_na
```

Top 10 words for topic #0

walk waterloo, also, zone, locate, central london, city, stay, restaurant, kitchen, thames, bedroom, location, flat, room, close, waterloo, river, view, central, apartment, station, minute walk, london, minute, walk

Top 10 words for topic #1

river thames, shop, central london, balcony, wharf, greenwich, restaurant, modern, kitchen, minute, central, river view, area, close, flat, station, park, apartment, thames, bedroom, room, view, london, river, walk

Top 10 words for topic #2

shop, restaurant, bathroom, park, wharf, apartment, private, distance, train, mile, river view, river thames, minute walk, mins, room, flat, minute, bedroom, close, view, thames, london, station, river, walk

10.2 Maximum Likelihood Topic

```
topic_values = LDA.transform(tcorpus)
topic_values.shape
```

```
(333, 3)
```

```
pd.options.display.max_colwidth=20
srcdf['Topic'] = topic_values.argmax(axis=1)
srcdf.head()
```

	geometry	listing_url	name	description	am
26	POINT (519474.79...	https://www.airb...	Kew Gardens 3BR ...	3 Bed House with...	[”C
626	POINT (530945.88...	https://www.airb...	BEST LOCATION in...	Newly renovated,...	[”C
962	POINT (544133.48...	https://www.airb...	Historic Royal A...	Our spacious, li...	[”C

	geometry	listing_url	name	description	am
1166	POINT (538098.29...	https://www.airb...	Spacious Charmin...	PLEASE READ OTHE...	["V
1208	POINT (531755.31...	https://www.airb...	Zone 1 City Dup...	A bright, spacio...	["C

```
pd.options.display.max_colwidth=75
srcdf[srcdf.Topic==1].description_norm.head(10)
```

```
962      spacious  light quiet apartment combine best charm london  juxtapositio...
3149     well decorate sunny garden flat fulham park gardens close putney bridge...
4455     sunny  quiet islington  bedroom flat . spacious  homely welcome . upper...
4813     room front house overlook balcony  road river . bright sunny house view...
6601     fabulous riverside flat  balcony great view thames canary wharf . next ...
6809     luxurious bedroom apartment zone love hidden secret part town  minute a...
6829     second room rent occasionally house  face rear overlook garden . listin...
7399     beautiful light cosy flat greenwich  short walk river thames amazing ri...
8230     overlooking river thames  zone london  next wapping station england cap...
10916    modern bright apartment timber floor . carpeted bedroom . leafy quiet a...
Name: description_norm, dtype: str
```

```
vectorizer = CountVectorizer(ngram_range=(1,1), stop_words='english', analyzer='word
topic_corpus = vectorizer.fit_transform(srcdf[srcdf.Topic==1].description.values) #
```

```
topicdf = pd.DataFrame(data=topic_corpus.toarray(),
                       columns=vectorizer.get_feature_names_out())
```

```
f,ax = plt.subplots(1,1,figsize=(5,5))
plt.gcf().set_dpi(300)
Cloud = WordCloud(
            background_color="white",
            max_words=75).generate_from_frequencies(topicdf.sum())
ax.imshow(Cloud)
ax.axis("off");
# plt.savefig('Wordcloud 3.png')
```



11 Word2Vec

> **Tip**
>
> This algorithm works almost like magic. You should play with the configuration parameters and see how it changes your results. **However**, at this time I've not preinstalled `gensim` as it bloated the image substantially.

11.1 Configure

```
from gensim.models.word2vec import Word2Vec
```

```
dims = 100
print(f"You've chosen {dims} dimensions.")

window = 4
print(f"You've chosen a window of size {window}.")

min_v_freq  = 0.01 # Don't keep words appearing less than 1% frequency
min_v_count = math.ceil(min_v_freq * srcdf.shape[0])
print(f"With a minimum frequency of {min_v_freq} and {srcdf.shape[0]:,} documents, m
```

11.2 Train

```
%%time

corpus      = srcdf.description_norm.fillna(' ').values
#corpus_sent = [nltk.sent_tokenize(text) for text in corpus] # <-- with more formal
corpus_sent = [d.replace('.',' ').split(' ') for d in corpus] # <-- deals better wit
model       = Word2Vec(sentences=corpus_sent, vector_size=dims, window=window, epoch
                min_count=min_v_count, seed=42, workers=1)

#model.save(f"word2vec-d{dims}-w{window}.model") # <-- You can then Word2Vec.load(..
```

11.3 Explore Similarities

This next bit of code only runs if you have calculated the frequencies above in the Frequencies and Ngrams section.

```
pd.set_option('display.max_colwidth',150)

df = fcounts[1] # <-- copy out only the unigrams as we haven't trained anything else

n     = 14 # number of words
topn  = 7  # number of most similar words
```

```
selected_words = df[df['Ngram Size 1'] > 5].reset_index().level_0.sample(n, random_s

words = []
v1    = []
v2    = []
v3    = []
sims  = []

for w in selected_words:
    try:
        vector = model.wv[w]  # get numpy vector of a word
        #print(f"Word vector for '{w}' starts: {vector[:5]}...")

        sim = model.wv.most_similar(w, topn=topn)
        #print(f"Similar words to '{w}' include: {sim}.")

        words.append(w)
        v1.append(vector[0])
        v2.append(vector[1])
        v3.append(vector[2])
        sims.append(", ".join([x[0] for x in sim]))
    except KeyError:
        print(f"Didn't find {w} in model. Can happen with low-frequency terms.")

vecs = pd.DataFrame({
    'Term':words,
    'V1':v1,
    'V2':v2,
    'V3':v3,
    f'Top {topn} Similar':sims
})

vecs
```

```
#print(model.wv.index_to_key) # <-- the full vocabulary that has been trained
```

11.4 Apply

We're going to make *use* of this further next week…

11.4.1 Questions

- What happens when *dims* is very small (e.g. 25) or very large (e.g. 300)?
- What happens when *window* is very small (e.g. 2) or very large (e.g. 8)?

12 Processing the Full File

> **Caution**
>
> This code can take *some time* (**> 5 minutes on a M2 Mac**) to run, so **don't run this** until you've understood what we did before!

You will get a warning about `"." looks like a filename, not markup` — this looks a little scary, but is basically suggesting that we have a description that consists only of a '.' or that looks like some kind of URL (which the parser thinks means you're trying to pass it something to download).

```
%%time
# This can take up to 8 minutes on a M2 Mac
gdf['description_norm'] = ''
gdf['description_norm'] = gdf.description.apply(normalise_document, remove_digits=Tr
```

```
gdf.to_parquet(os.path.join('data','geo',f'{fn.replace(".","-with-nlp.")}'))
```

> **Tip**
>
> Saving an intermediate file at this point is useful because you've done quite a bit of *expensive* computation. You *could* restart-and-run-all and then go out for the day, but probably easier to just save this output and then, if you need to restart your analysis at some point in the future, just remember to deserialise amenities back into a list format.

12.1 Applications

The above is *still* only the results for the one of the subsets of apartments *alone*. At this point, you would probably want to think about how your results might change if you changed any of the following:

1. Using one of the other data sets that we created, or even the entire data set!
2. Applying the CountVectorizer or TfidfVectorizer *before* selecting out any of our 'sub' data sets.
3. Using the visualisation of information to improve our regex selection process.
4. Reducing, increasing, or constraining (i.e. `ngrams=(2,2)`) the size of the ngrams while bearing in mind the impact on processing time and interpretability.
5. Filtering by type of listing or host instead of keywords found in the description (for instance, what if you applied TF/IDF to the entire data set and then selected out 'Whole Properties' before splitting into those advertised by hosts with only one listing vs. those with multiple listings?).
6. Linking this back to the geography.

Over the next few weeks we'll also consider alternative means of visualising the data!

12.2 Resources

There is a lot more information out there, including a whole book and your standard O'Reilly text.

And some more useful links:

- Pandas String Contains Method
- Using Regular Expressions with Pandas
- Summarising Chapters from Frankenstein using TF/IDF